

CORPORACIÓN UNIVERSITARIA REMINGTON
UNIREMINGTON

ARQUITECTURA ORIENTADA A MICROSERVICIOS

Ejercicio práctico de comunicación entre servicios mediante API REST

Omar Antonio Mosquera Palacios

Lenguaje de Programación Avanzado 2

20 de septiembre de 2026

Resumen

Este trabajo presenta una aproximación teórica y práctica a la arquitectura orientada a microservicios. Se comparan sus características con las de una arquitectura monolítica y se implementan dos servicios independientes con Java 17 y Spring Boot 3.2.5. El servicio de pedidos consulta al servicio de usuarios mediante una API REST, permitiendo observar de manera sencilla cómo se comunican componentes distribuidos.

Palabras clave: microservicios, arquitectura de software, Spring Boot, API REST, servicios distribuidos.

1. Introducción

Las aplicaciones modernas requieren arquitecturas capaces de evolucionar, escalar y mantenerse de forma organizada. La arquitectura de microservicios propone dividir una aplicación en servicios pequeños e independientes, organizados alrededor de capacidades de negocio. Cada servicio puede desarrollarse, probarse y desplegarse de manera independiente, mientras se comunica con otros servicios mediante interfaces bien definidas.

En esta actividad se estudian las diferencias entre una arquitectura monolítica y una arquitectura de microservicios y se desarrolla un ejemplo sencillo utilizando Spring Boot. El ejercicio se concentra en dos servicios: Usuarios y Pedidos.

2. Objetivos

- Reconocer los principios básicos de la arquitectura de microservicios.
- Diferenciar una arquitectura monolítica de una arquitectura orientada a microservicios.
- Identificar ventajas, desventajas y escenarios de aplicación.
- Implementar un ejemplo básico de comunicación entre servicios mediante una API REST.

3. Cuadro comparativo

Criterio	Monolítica	Microservicios
Estructura	Una aplicación concentra los módulos principales.	El sistema se divide en servicios independientes.
Despliegue	Generalmente se despliega la aplicación completa.	Los servicios pueden desplegarse de manera independiente.
Escalabilidad	Normalmente se escala la aplicación como un todo.	Se puede escalar un servicio específico según la demanda.
Comunicación	Principalmente mediante llamadas internas.	Mediante APIs, HTTP, mensajería u otros mecanismos.
Mantenimiento	Un cambio puede afectar diferentes módulos.	Cada servicio puede evolucionar con mayor independencia.
Tecnologías	Suele existir un conjunto tecnológico común.	Los servicios pueden utilizar tecnologías diferentes.
Complejidad	Menor complejidad inicial.	Mayor complejidad operativa y de comunicación.
Datos	Es frecuente una base de datos centralizada.	Los servicios pueden administrar sus propios datos.

4. Ensayo: adopción de microservicios en aplicaciones modernas

La arquitectura de microservicios se ha consolidado como una alternativa para sistemas que necesitan evolucionar con rapidez y manejar diferentes niveles de demanda. En este enfoque, una aplicación se organiza como un conjunto de servicios pequeños, cada uno con una responsabilidad definida y con comunicación mediante interfaces explícitas. Esta separación puede facilitar el trabajo de equipos que mantienen componentes diferentes del mismo producto.

Una característica importante es la posibilidad de realizar cambios y despliegues de manera independiente. En una aplicación monolítica, una modificación en un componente puede requerir construir y desplegar toda la aplicación. En una arquitectura de microservicios, el cambio puede limitarse al servicio correspondiente cuando el diseño y las dependencias lo permiten.

La escalabilidad también es una razón frecuente para considerar este enfoque. Si un componente recibe una carga significativamente mayor que otros, es posible aumentar los recursos de ese servicio sin necesariamente replicar toda la aplicación.

Sin embargo, la adopción de microservicios introduce nuevos retos. La comunicación deja de ser únicamente interna y pasa a depender de la red, por lo que aparecen aspectos como latencia, disponibilidad, autenticación, manejo de errores, observabilidad y coordinación de versiones. También aumenta el número de componentes que deben desplegarse y monitorearse.

Por lo anterior, los microservicios no deben considerarse una solución universal. Su conveniencia depende del tamaño, las necesidades de escalabilidad, la frecuencia de cambios, la estructura de los equipos y la capacidad técnica disponible. Para una aplicación pequeña, un monolito puede ser suficiente; para sistemas con componentes que requieren evolución y escalamiento independiente, los microservicios pueden ofrecer ventajas importantes.

5. Desarrollo de la actividad práctica

Se desarrollaron dos aplicaciones independientes utilizando Java 17 y Spring Boot 3.2.5. El primer proyecto corresponde a **usuarios-service** y se ejecuta en el puerto 8081. El segundo corresponde a **pedidos-service** y se ejecuta en el puerto 8082.

El servicio de usuarios expone los endpoints GET /usuarios y GET /usuarios/{id}. El servicio de pedidos expone GET /pedidos y GET /pedidos/{id}. Cuando se solicita un pedido por su identificador, pedidos-service consulta mediante HTTP al servicio de usuarios para obtener los datos del usuario relacionado.

Esta implementación permite observar una característica esencial de los microservicios: los servicios son procesos independientes y su integración se realiza mediante una interfaz de comunicación definida.

6. Diagrama de arquitectura

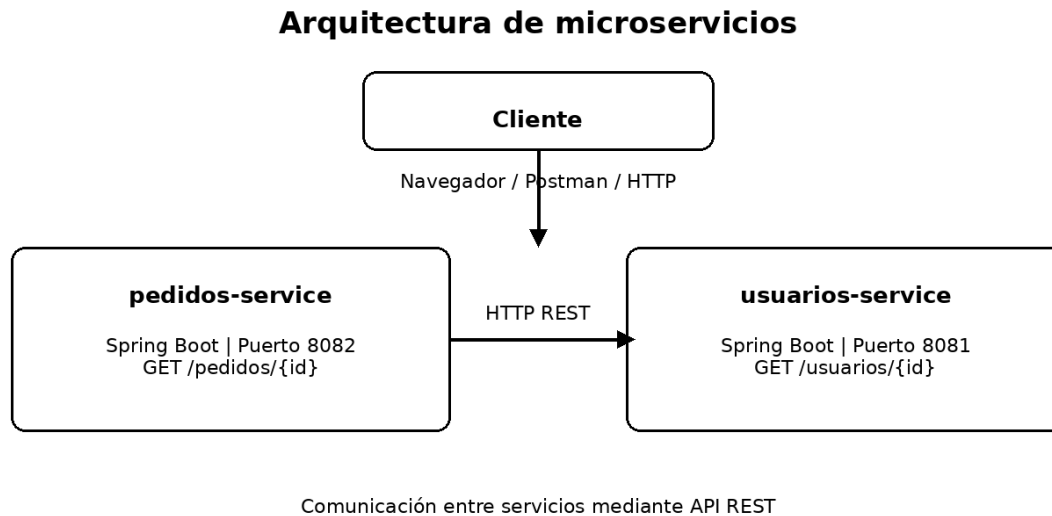


Figura 1. Arquitectura propuesta para la comunicación entre los servicios. Elaboración propia.

7. Endpoints y funcionamiento

Usuarios: GET <http://localhost:8081/usuarios> y GET <http://localhost:8081/usuarios/1>.

Pedidos: GET <http://localhost:8082/pedidos> y GET <http://localhost:8082/pedidos/1>.

La prueba principal consiste en consultar `/pedidos/1`. El controlador de pedidos obtiene el pedido y utiliza un cliente HTTP para solicitar al servicio de usuarios la información asociada.

8. Código y organización

El proyecto se organiza en dos aplicaciones Spring Boot independientes. Cada una contiene su clase principal, controladores y modelos. El servicio de pedidos incluye además un cliente para realizar la llamada REST al servicio de usuarios. La separación permite identificar con claridad la responsabilidad de cada componente.

9. Resultados esperados de la ejecución

Al ejecutar los dos servicios simultáneamente deben quedar disponibles los puertos 8081 y 8082. La consulta de usuarios debe devolver información de usuario y la consulta de pedidos debe devolver información del pedido junto con la información obtenida desde usuarios-service. La

sustentación en video mostrará la ejecución real de estas pruebas.

10. Conclusiones

La actividad permitió comprender que los microservicios dividen una aplicación en componentes independientes con responsabilidades concretas. Esta separación facilita determinados escenarios de evolución y escalabilidad, aunque introduce retos relacionados con la comunicación distribuida y la operación del sistema.

El ejercicio práctico permitió comprobar el concepto mediante dos servicios Spring Boot. La comunicación REST entre Pedidos y Usuarios representa un ejemplo básico de integración entre microservicios y permite relacionar los conceptos teóricos con una implementación ejecutable.

11. Entregables y enlaces

Código fuente: repositorio GitHub del proyecto. La URL se incorpora al publicar el repositorio de entrega.

Video de sustentación: video de máximo 10 minutos mostrando la ejecución, funcionamiento y explicación del desarrollo. La URL se incorpora al publicar la sustentación.

El código fuente, el diagrama y el guion se encuentran organizados dentro de la carpeta del proyecto.

12. Referencias

Fowler, M. (2014). *Microservices*. Martin Fowler. <https://martinfowler.com/articles/microservices.html>

Newman, S. (2021). *Building microservices* (2nd ed.). O'Reilly Media.

Spring. (s. f.). *Spring Boot reference documentation*. <https://docs.spring.io/spring-boot/docs/current/reference/html/>

Spring. (s. f.). *Building a RESTful web service*. <https://spring.io/guides/gs/rest-service/>

Corporación Universitaria Remington. (2026). *Uniremington*. <https://www.uniremington.edu.co/>

Presentación basada en criterios de APA 7.^a edición: portada, jerarquía de títulos, tipografía Times New Roman, interlineado 1,5, numeración de páginas y referencias.